

COMPREHENSIVE GAP ANALYSIS

"Malware Detection with Artificial Intelligence: A Systematic Literature Review" (2024)

Complete Inventory of Missing Topics, Methodological Gaps, & Critical Weaknesses

EXECUTIVE SUMMARY: CRITICAL GAPS IDENTIFIED

The paper demonstrates **significant structural and content gaps** that limit its utility as a decade-spanning (2015–2025) survey:

CRITICAL GAPS (Publication-Stopping Issues):

- **✗ Zero coverage** of transformer architectures & modern LLMs
- **✗ Absent** graph neural network methodologies
- **✗ Missing** EMBER2024 (major 2025 benchmark)
- **✗ No discussion** of adversarial robustness against GAN-based evasion
- **✗ Minimal** explainability/interpretability methods (SHAP, LIME, Grad-CAM)
- **✗ Absent** concept drift & temporal evaluation frameworks
- **✗ No coverage** of cross-dataset generalization evaluation
- **✗ Missing** binary lifting & IR-based semantic analysis

MAJOR GAPS (Reduce Quality/Completeness):

- **✗ Weak** discussion of practical deployment & scalability realities
- **✗ No** reproducibility guidance (tool versioning, containerization)
- **✗ Minimal** few-shot & meta-learning coverage
- **✗ Absent** edge computing & lightweight model deployment
- **✗ Limited** supply-chain malware & APT analysis
- **✗ Missing** reinforcement learning approaches

METHODOLOGICAL GAPS (Affect Survey Validity):

- **✗ Publication cutoff** ~Sep 2022 (2+ years outdated for fast-moving field)
- **✗ No temporal stratification** of papers analyzed
- **✗ Lack of critical analysis** of why techniques evolved
- **✗ No systematic evaluation** of generalization across datasets
- **✗ Missing** failure mode analysis for each technique
- **✗ Limited** discussion of practical vs. research-only approaches

STRUCTURAL GAPS (Impact Readability/Utility):

- **✗ No visual timeline** showing technique evolution
 - **✗ No maturity matrix** for evaluating when to use which method
 - **✗ Missing reading list** for practitioners/students
 - **✗ No from-scratch writing guide** for future survey authors
 - **✗ Weak guidance** on what readers should implement today
 - **✗ No unified taxonomy** showing paradigm shifts
-

I. TECHNIQUE COVERAGE GAPS (Detailed)

A. Graph Neural Networks & Structural Analysis

Current Status in Paper: Minimal (~2 mentions of CFGExplainer [4]; not featured)

Gap Severity: CRITICAL — GNNs represent SOTA for graph-based malware analysis (2019–2025)

Missing Content:

1. GNN Architectures Not Covered:

- Graph Convolutional Networks (GCN) on Control Flow Graphs (CFG)
- Graph Attention Networks (GAT) for feature importance
- Graph Isomorphism Networks (GIN) for robustness
- GraphSAGE for sampling-based graph learning
- **Evidence of gap:** [web:4], [web:7] show GNN applications; paper minimal

2. CFG/DFG Analysis:

- Dynamic CFG extraction from binary
- Graph-based similarity detection
- Structural invariance to polymorphism
- **Evidence:** BE-PUM [68] mentioned briefly; no GNN integration discussed

3. Graph Embedding Methods:

- node2vec, DeepWalk on CFG nodes
- Contrastive learning on graph representations
- **Missing entirely**

4. GNN Adversarial Robustness:

- Can GNNs withstand graph perturbations (opcode insertion, CFG flattening)?
- Certified robustness on graphs?

- **Evidence:** [web:10] shows active opcode insertion defeating GNNs; not discussed

5. Explainability on Graphs:

- Which subgraph motifs indicate malware?
- CFGExplainer [web:4] highlights interpretable nodes
- **Missing entirely from paper**

Recommendation: Add 3–4 pages on GNN methods; cite recent work on graph-based malware detection; discuss CFG robustness

B. Binary Lifting & Intermediate Representations

Current Status in Paper: **Completely absent** (0 mentions)

Gap Severity: CRITICAL — IR-based analysis is modern approach to obfuscation robustness

Missing Content:

1. Binary Lifting Frameworks:

- Valgrin IR semantics
- BAP (BitBlaze Analysis Platform)
- LLVM IR as analysis target
- Retargetable Decompiler (RTD)
- **Evidence:** [web:14], [web:17], [web:20] — established literature

2. Semantic-Robust Feature Extraction:

- IR-level opcode sequences (invariant to syntactic obfuscation)
- Decompiled pseudocode understanding
- Cross-architecture IR for generalization (x86 → ARM → MIPS)
- **Gap:** Paper only discusses syntactic features (opcode n-grams); no semantic layer

3. IR-Based Malware Detection:

- Training classifiers on IR rather than assembly
- Robustness to polymorphism/metamorphism via IR invariance
- **Missing entirely**

4. Lifting Correctness & Reproducibility:

- [web:20] found 24 semantic bugs in state-of-the-art lifters
- Impact on feature quality
- Tool version differences in Ghidra/IDA
- **Paper ignores this critical issue**

5. Decompilation Quality:

- Pseudocode recovery accuracy

- Proprietary vs. open-source decompilers
- LLM understanding of decompiled code
- **Completely absent**

Recommendation: Add 2–3 pages on IR-based methods; explain why semantic analysis is more robust; discuss tool correctness

C. Transformer Architectures & Self-Attention

Current Status in Paper: **Not dedicated section** (only LLM mention in final page)

Gap Severity: CRITICAL — Transformers are now SOTA for sequence analysis (2023–2025)

Missing Content:

1. Transformer Basics for Malware:

- Why transformers (parallel, long-range dependencies) outperform RNNs
- Self-attention mechanisms on opcode sequences
- Positional encoding for instruction sequences
- **Paper treats this as background knowledge; doesn't explain paradigm shift**

2. Specific Transformer Models:

- **BinBert** [web:3]: Execution-aware transformer for assembly
 - Pre-trained on large assembly corpus
 - Fine-tuned on malware tasks
 - SOTA on multiple benchmarks
 - **Paper: zero mention**
- **Vision Transformers** [web:5]: Binary images → transformer
 - Treats malware as images; applies ViT
 - **Paper: zero mention**
- **Process-Resource Transformers** [web:11]: API/registry → transformer
 - Dynamic behavior → transformer encoding
 - **Paper: zero mention**

3. Pre-Training & Transfer Learning:

- Self-supervised pre-training on unlabeled assembly
- Fine-tuning on labeled malware (reducing feature engineering)
- Task-specific adapters (detection, classification, behavior)
- **Completely absent from paper**

4. Transformer vs. CNN/RNN Comparison:

- When is transformer preferable?
- Computational trade-offs (larger models, more VRAM)
- Interpretability challenges
- **Not addressed**

5. Attention Weight Analysis:

- Which instructions are "important" for malware?
- Attention-based explanations
- **Missing entirely**

Recommendation: Add 4–5 pages on transformers; explain why they're SOTA; cite BinBert, Vision Transformers; discuss when to use vs. GNNs

D. LLM-Assisted & Foundation Model Analysis

Current Status in Paper: 4 papers total; brief LLM mention; not integrated into methodology

Gap Severity: CRITICAL — LLMs represent paradigm shift in binary understanding (2023–2025)

Missing Content:

1. LLM-Based Semantic Understanding:

- CodeLLaMA, GPT-4, Gemini understanding of assembly
- Prompt-based malware analysis (e.g., "explain the intent of this code")
- Semantic lifting without manual decompilation
- **Evidence:** [web:24], [web:27] — significant work on LLM code understanding
- **Paper:** minimal coverage

2. Foundation Model Architecture:

- Large pre-trained models on code corpora
- Few-shot/zero-shot learning for novel malware
- Transfer learning from code LLMs to binary domain
- **Completely missing**

3. LLM-Based Decompilation:

- LLM improving decompilation quality
- Reconstructing high-level logic from assembly
- **Not addressed**

4. Adversarial Robustness of LLMs:

- Can malware fool LLM detectors via adversarial inputs?

- Prompt injection attacks on LLM-based analysis
- Jailbreaking LLM safety measures
- **Critical gap:** paper doesn't address threats to LLM-based approaches

5. Efficiency & Edge Deployment:

- LLMs are expensive (GPU-heavy)
- Can lightweight LLMs run on edge? [web:40]
- Quantization, distillation for deployment
- **Paper ignores practical constraints**

6. Interpretability of LLM Decisions:

- Why did LLM classify as malware?
- Chain-of-thought prompting for explanations
- **Missing entirely**

Recommendation: Add 5-6 pages dedicated to LLM-assisted analysis; discuss architecture, advantages, limitations, threats

E. Explainability & Interpretability Methods

Current Status in Paper: Zero coverage (not mentioned once)

Gap Severity: HIGH — Critical for production deployment & regulatory compliance

Missing Content:

1. SHAP (SHapley Additive exPlanations):

- Feature importance calculation
- Local & global explanations
- Applied to malware detectors
- **Evidence:** [web:22], [web:25] — comprehensive XAI studies 2024-2025
- **Paper:** not mentioned

2. LIME (Local Interpretable Model-Agnostic Explanations):

- Local approximations of classifier decisions
- Why was sample X classified as malware?
- **Missing entirely**

3. Grad-CAM & Attention-Based Explainability:

- Gradient-based feature importance
- Attention weight visualization
- **Not addressed**

4. Integrated Gradients & Saliency Maps:

- Attribution methods for neural networks
- Which opcodes drove detection?
- **Missing**

5. Counterfactual Explanations:

- "If opcode sequence X were Y, would it still be detected?"
- Minimal modification for explanation
- **Not discussed**

6. Trade-offs: Accuracy vs. Interpretability:

- Do interpretable models sacrifice accuracy?
- Surrogate model approach
- **Missing**

Recommendation: Add 2-3 pages on XAI for malware; cite [web:22], [web:25], [web:31]; explain why explainability matters for security

F. Few-Shot & Meta-Learning

Current Status in Paper: Completely absent (zero mentions)

Gap Severity: HIGH — Critical for novel/zero-day malware detection

Missing Content:

1. Few-Shot Learning Paradigm:

- Rapid adaptation to new malware families with <10 samples
- Prototypical networks, matching networks
- **Evidence:** [web:23] — SIMPLE few-shot meta-learning for malware
- **Paper:** not mentioned

2. Meta-Learning Approaches:

- MAML (Model-Agnostic Meta-Learning) for malware
- Task-aware feature learning
- **Missing entirely**

3. Zero-Shot Learning:

- Detecting malware from completely unseen families without labeled data
- Attribute-based or semantic transfer
- **Not addressed**

4. Transfer Learning from Code LLMs:

- Fine-tuning CodeLLaMA on malware tasks

- Parameter-efficient adaptation (LoRA, adapters)
- **Completely absent**

5. Evaluation on Unseen Families:

- Papers claiming "zero-day detection": how are they evaluating?
- Hold-out family evaluation protocol
- **Paper doesn't discuss evaluation methodology**

Recommendation: Add 2 pages on few-shot & meta-learning; cite [web:23]; explain when useful for zero-day detection

G. Reinforcement Learning & Active Learning

Current Status in Paper: Minimal coverage (active learning mentioned once [48])

Gap Severity: MEDIUM — Emerging techniques not yet dominant

Missing Content:

1. Reinforcement Learning for Malware Detection:

- Adversary learning malware → detector learning defense
- Game-theoretic approaches
- **Not discussed**

2. Active Learning:

- Intelligently selecting samples for human labeling
- Reduces annotation burden
- **Barely mentioned; not analyzed**

3. Curriculum Learning:

- Training on easy samples → hard samples
- Progressive difficulty for malware detection
- **Missing**

4. Online Learning:

- Model updates continuously as new malware arrives
- Concept drift adaptation in real-time
- **Not addressed**

Recommendation: Add 1 page on RL/active learning; note these are emerging; cite relevant work if available

H. Semantic-Level Analysis & Program Understanding

Current Status in Paper: Weakly covered (implicit in "features")

Gap Severity: MEDIUM — Emerging paradigm for robustness

Missing Content:

1. Symbolic Execution:

- Exploring all execution paths statically
- Constraint solving for reachability
- Applied to malware detection
- **Paper mentions but doesn't develop**

2. Taint Analysis:

- Tracking data flow from sources (file reads) to sinks (network sends)
- Malware behavior characterization
- **Missing**

3. Type-Based Analysis:

- Type inference in binaries
- Structural types revealing intent
- **Not addressed**

4. Semantic Equivalence Detection:

- Two binaries with same semantic behavior but different syntax
- Robust to obfuscation
- **Missing entirely**

Recommendation: Add 1-2 pages on semantic analysis; explain robustness benefits; note this is emerging

II. DATASET & EVALUATION GAPS (Detailed)

A. EMBER2024 Completely Missing

Current Status in Paper: Not mentioned (paper published Jan 2024, released Jun 2025)

Gap Severity: CRITICAL — Major new benchmark with significant advances

Missing Content:

1. What is EMBER2024:

- 3.2M samples (vs. EMBER 1.1M)
- 6 file formats (PE, ELF, Mach-O, DEX, APK, etc.)
- 7 tasks (not binary classification):
 - Malware detection
 - Family classification

- Behavior identification
- Others
- Evasion-focused "challenge set"
 - Samples initially undetected by ≥ 3 AV products
 - Tests realistic robustness
- Published baseline models
- **Evidence:** [web:15], [web:21] (Jun 2025 release)
- **Paper:** zero mention

2. Feature Version 3:

- Extended features beyond EMBER v2
- New static/dynamic types
- **Not discussed**

3. Multi-Task Learning Benchmark:

- Single dataset enables multiple tasks
- Like SuperGLUE for NLP
- **Paper missing this paradigm**

4. Cross-File-Format Evaluation:

- Does model trained on PE generalize to ELF/APK?
- **Not addressed**

Recommendation: Add section on EMBER2024; update dataset discussion; emphasize evasion-focused evaluation

B. Concept Drift & Temporal Evaluation Framework

Current Status in Paper: Acknowledged as "challenge"; no evaluation methodology provided

Gap Severity: HIGH — Critical for real-world deployment

Missing Content:

1. Concept Drift Definition & Characterization:

- Distribution shift in malware features over time
- How fast does drift occur? (months? weeks?)
- **Paper acknowledges but doesn't quantify**

2. Temporal Evaluation Methodology:

- Time-stratified train/test splits (not random split)
- Train on 2020, test on 2021, 2022, 2023, etc.
- Measure accuracy degradation over time
- **Paper mentions BODMAS timestamp awareness; no evaluation protocol**

3. Empirical Drift Studies:

- LAMD [web:32]: Models trained on 2008-2012 fail on 2017-2020
- F1 drops from 0.9 to 0.6
- How bad is drift in malware domain?
- **Paper doesn't quantify drift magnitude**

4. Adaptive/Continual Learning:

- MADCAT [web:38]: Test-time training with pseudo-labeling
- Warm-start learning for drift
- **Completely absent from paper**

5. Concept Drift Detection:

- How to detect drift without labels?
- Performance metrics can lag actual distribution shift
- **Not addressed**

Recommendation: Add 2-3 pages on temporal evaluation; recommend time-stratified benchmark design; cite drift studies

C. Cross-Dataset Generalization Evaluation

Current Status in Paper: Generalization failures noted; no systematic evaluation protocol

Gap Severity: HIGH — Critical for production applicability

Missing Content:

1. Cross-Dataset Protocol:

- Standard: Train on EMBER, test on SOREL-20M, VirusShare, BODMAS, Malware Bazaar
- Measure domain gap
- **Not recommended in paper; no standard protocol**

2. Dataset Differences:

- Temporal windows (EMBER 2017-2018 vs. SOREL 2017-2019)
- Feature extraction method differences
- Labeling methodology (VirusTotal consensus vs. manual)
- **Paper doesn't systematically analyze differences**

3. Domain Adaptation Techniques:

- Transfer learning to new dataset
- Adversarial domain adaptation
- **Evidence:** [web:33], [web:35] show domain adaptation importance
- **Paper:** minimal coverage

4. Benchmarking Best Practices:

- Should papers train/test on same dataset?
- How to publish results fairly?
- **Not discussed**

5. Feature Reuse Across Datasets:

- Are EMBER features extracted same way on SOREL-20M?
- Tool versioning issues
- **Missing**

Recommendation: Add 2 pages on cross-dataset evaluation; recommend standard protocol; discuss domain adaptation

D. Adversarial Evaluation & Robustness Testing

Current Status in Paper: Anti-analysis techniques covered; detector robustness minimal

Gap Severity: HIGH — Critical for secure deployment

Missing Content:

1. Adversarial Example Generation:

- GAN-based malware generation [web:13]
- Gradient-based perturbations (FGSM, PGD)
- Genetic algorithms for evasion
- **Paper discusses evasion; not adversarial ML evaluation**

2. Certified Robustness:

- Provable guarantees against perturbations within bound
- Interval Bound Propagation (IBP)
- Randomized smoothing
- **Completely absent from paper**

3. Threat Model Definition:

- What perturbations are valid? (malware must remain functional)
- Which features can attacker modify?
- **Not addressed**

4. Evaluation Metrics for Robustness:

- Clean accuracy vs. robust accuracy
- NDCG (Normalized Discounted Cumulative Gain) for ranking robustness
- **Not mentioned**

5. Adversarial Training:

- Training on adversarial examples
- Accuracy-robustness trade-off
- **Barely mentioned; not analyzed**

6. GAN-Based Evasion Studies:

- MalGAN, DOpGAN, other adversarial malware
- How well do they defeat modern detectors?
- **Evidence:** [web:13] comprehensive 2025 review
- **Paper:** minimal coverage

Recommendation: Add 3 pages on adversarial robustness; cite [web:13]; discuss evaluation framework

E. False Positive Rate & Production-Relevant Metrics

Current Status in Paper: Uses accuracy, precision, recall, F1; limited FPR discussion

Gap Severity: MEDIUM — Important for production deployment

Missing Content:

1. FPR at Production Thresholds:

- Report FPR at TPR=95%, TPR=99%
- FPR=1/1000, FPR=1/10,000 (production constraints)
- **Paper uses AUC; doesn't focus on useful operating points**

2. Imbalanced Dataset Metrics:

- ROC curves focus on FPR; misleading with imbalanced data
- Precision-recall curves more informative
- **Paper doesn't discuss**

3. Cost Matrix:

- Cost of false positive (block benign file)
- Cost of false negative (miss malware)
- **Paper mentions CSPE-R [105] uses cost matrix; not systematized**

4. EMBER Saturation Issue:

- Harang & Rudd: Baseline AUC 0.999 on EMBER
- Suggests benchmark too easy
- **Paper acknowledges; doesn't emphasize severity**

5. Comparison Across Operating Points:

- Don't just report "98% accuracy"

- Report full curves; compare at relevant FPR
- **Not done in paper**

Recommendation: Add 1 page on metric selection; explain production relevance; criticize accuracy-only reporting

III. METHODOLOGY & EVALUATION GAPS

A. Publication Cutoff Too Recent for "State-of-the-Art"

Current Status: Paper published Jan 2024; literature cutoff Sep 2022-Sep 2023

Gap Severity: CRITICAL — Major omissions of 2024-2025 work

Issues:

- Transformers (BinBert late 2023) mostly missed
- LLMs (2023-2025 explosion) minimally covered
- EMBER2024 (Jun 2025) completely absent
- Concept drift studies (2023-2025) sparse
- Adversarial robustness evolution (2024-2025) not captured

Recommendation: Extend cutoff to Dec 2024 minimum; add major 2025 papers retrospectively

B. No Temporal Stratification of Papers Analyzed

Current Status: Papers analyzed; no distribution table by publication year

Gap Severity: MEDIUM — Makes temporal trends invisible

Issues:

- Can't see which techniques were dominant in which era
- Can't identify inflection points
- Conflates 2015 DL approaches with 2023 transformers
- **Solution:** Create table: Year × Technique × Paper Count

Recommendation: Add temporal breakdown table; identify technique adoption curves

C. No Failure Mode Analysis

Current Status: Techniques discussed in isolation; limitations noted briefly

Gap Severity: MEDIUM — Reduces actionability

Issues:

- Why do CNN on binaries fail at scale? (No discussion)
- Why do static-only detectors fail on obfuscated malware? (Noted; not analyzed deeply)
- Why does hybrid approach still underperform? (Not addressed)

Recommendation: Add "Failure Modes" subsection for each major technique; explain why failures occur

D. No "Gotchas" or Practical Pitfalls

Current Status: Paper presents methods neutrally; doesn't warn about pitfalls

Gap Severity: MEDIUM — Graduate students may repeat mistakes

Missing Warnings:

- EMBER dataset saturation (Harang & Rudd warning [37])
- Dataset imbalance affecting metrics
- Feature leakage in train/test splits
- Tool bugs affecting feature extraction [web:20]
- Benign software using same APIs as malware [59]
- DBI overhead preventing real-world analysis

Recommendation: Add "Practical Pitfalls" section warning researchers of common mistakes

IV. PRACTICAL DEPLOYMENT & SCALABILITY GAPS

A. Scalability to Real-World Volumes

Current Status in Paper: Mentioned that DBI is slow; no formal scalability analysis

Gap Severity: HIGH — Critical for production deployment

Missing Content:

1. Real-World Throughput Requirements:

- AV Test: 450,000 new samples/day
- Commercial detectors: millions/day
- Paper mentions this stat; doesn't address analysis bottleneck
- **Can we afford dynamic analysis for all files?**

2. Feature Extraction Bottleneck:

- Static analysis: <1 sec/file
- Cuckoo sandbox: 30-60 sec/file
- DBI: 1-5 sec/file (but high overhead)
- Paper mentions; no systematic comparison

3. Tiered Pipeline Economics:

- Tier 1 (fast static, all files): ~1M files/day
- Tier 2 (dynamic, suspicious): ~10K files/day
- Tier 3 (manual, critical): ~100 files/day
- **Paper doesn't design tiered approach**

4. Cost-Benefit Analysis:

- Detection rate vs. computational cost
- When is dynamic analysis worth it?
- **Not analyzed**

5. Cloud vs. On-Premises Trade-offs:

- GPU costs for DL models
- Latency constraints
- **Not addressed**

Recommendation: Add section on scalability; discuss tiered pipelines; estimate costs

B. Inference Latency & Resource Requirements

Current Status in Paper: AI-Hydra latency mentioned (60.9 sec); no systematic comparison

Gap Severity: MEDIUM — Important for deployment

Missing Content:

1. Model Comparison Table:

- Model type, inference latency, memory, GPU requirement, throughput
- **Table not provided**

2. Latency Distribution:

- Percentiles (p50, p95, p99)
- Tail latency matters for interactive use
- **Not discussed**

3. Hardware Accelerators:

- Can GPUs/TPUs/specialized accelerators help?

- Cost-benefit
- **Not addressed**

4. Model Compression:

- Distillation, quantization for lightweight deployment
- Accuracy-latency trade-off
- **Mentioned for TensorFlow Lite [25]; not systematized**

Recommendation: Add table on inference cost; recommend profiling benchmarks

C. Tool Versioning & Reproducibility

Current Status in Paper: Tools mentioned (IDA, Ghidra, Cuckoo, Pin); no reproducibility guidance

Gap Severity: HIGH — Critical for scientific rigor

Missing Content:

1. Tool Version Differences:

- IDA Pro 7.x vs. 8.x disassembly differences
- Ghidra 10.0 vs. 11.0 decompilation changes
- Cuckoo 2.0 vs. 3.0 feature extraction differences
- **Paper ignores versioning issues**

2. Disassembler Inconsistencies:

- Evidence [web:20]: 24 semantic bugs in binary lifters
- Different tools produce different assembly
- **Critical gap not addressed**

3. Feature Extraction Reproducibility:

- Same binary → different features if tools differ
- How to ensure reproducibility?
- **Not discussed**

4. Containerization & Automation:

- Docker for reproducible environments
- Scripts for automated analysis
- **Not mentioned**

5. Open-Source vs. Proprietary Tools:

- IDA Pro (expensive, not reproducible)
- Ghidra (free, open-source, recommended for reproducibility)
- **Paper doesn't recommend open-source**

Recommendation: Add reproducibility section; recommend Ghidra + open-source tools; emphasize version pinning

D. Hybrid & Cascading Pipeline Design

Current Status in Paper: Hybrid static+dynamic discussed (AI-Hydra, CSPE-R); not framed as pipeline

Gap Severity: MEDIUM — Important for practical deployment

Missing Content:

1. Pipeline Architecture: ``` Tier 1 (Fast Static):

- PE headers, strings, opcode n-grams
- <1 second
- Fast reject for obvious benign
- Fast accept for obvious malware

Tier 2 (If Uncertain):

- Cuckoo sandbox or DBI
- 30-60 seconds
- Deeper behavioral analysis

Tier 3 (If Still Uncertain):

- Human analyst
- Symbolic execution, manual review ```
 - **Paper mentions hybrid; doesn't design cascade**

2. Decision Thresholds:

- When to escalate to next tier?
- Cost-benefit of uncertainty
- **Not addressed**

3. Operational Considerations:

- Parallel processing for scalability
- Fallback if tier fails
- **Not discussed**

Recommendation: Add 1-2 pages on cascading pipelines; design realistic deployment architecture

E. Static Analysis Viability Under Obfuscation

Current Status in Paper: Obfuscation prevalence noted (60-80%); questions feasibility

Gap Severity: HIGH — Fundamental to survey validity

Issues:

- If 60–80% of malware use obfuscation
- And static analysis is defeated by obfuscation [59]
- Then **is static analysis viable for production?**
- **Paper acknowledges but doesn't boldly state: "Static analysis is becoming obsolete"**

Missing:

- **Recommendation:** Use IR-based or semantic analysis instead of syntactic
- **Guidance:** When to use dynamic analysis despite cost
- **Alternative:** LLM-based semantic understanding

Recommendation: Add critical discussion of static analysis viability; recommend semantic alternatives

V. LEARNING PARADIGM & EVOLUTION GAPS

A. No Clear Stratification of Legacy vs. Current Methods

Current Status: Techniques discussed; not labeled as "legacy" vs. "SOTA"

Gap Severity: MEDIUM — Confuses readers on what to implement

Issues:

- Signature-based detection (pre-2010) discussed equally with deep learning (2015+)
- No guidance: "If starting today, skip signature-based; start with DL"
- Early DL (CNN/RNN vanilla 2015–2017) not clearly marked as superseded
- Figure 4 conflates "ML" and "DL" without sub-paradigms

Missing Table: Paradigm Era Legacy? Baseline? SOTA? Recommendation

Signature Pre-2010 YES – – Skip Classical ML 2010–2017 NO YES – Use if lightweight/fast Early DL 2015–2018 NO – – Don't use; superseded Hybrid 2018–2021 NO YES – Use if resources available GNN 2019–2023 NO – YES Use for structural data Transformer 2023–2025 NO – YES Use if resources available LLM-Assisted 2023–2025 NO – EMERGING Use if semantic

- **Table not provided in paper**

Recommendation: Add paradigm stratification table; clearly label what to use today

B. No Discussion of Why Paradigms Shifted

Current Status: Techniques presented; no "why this emerged" narrative

Gap Severity: MEDIUM — Reduces understanding of technique landscape

Missing Explanations:

1. Why GNNs (2019-2021)?

- Because classical features lose structural info
- GNNs preserve CFG/DFG topology
- Better robustness to polymorphism
- **Paper doesn't explain motivation**

2. Why Transformers (2023-2025)?

- Because RNNs suffer vanishing gradients on long sequences
- Transformers parallel, scale better, pre-train more effectively
- **Not explained**

3. Why LLMs (2023-2025)?

- Because feature engineering burden is high
- LLMs carry semantic knowledge from code corpora
- Transfer learning reduces labeling requirement
- **Not discussed**

4. Why IR-Based Semantics?

- Because syntactic obfuscation defeats static analysis
- IR-level semantics more invariant to polymorphism
- **Completely absent**

Recommendation: Add "Evolution Narrative" section explaining why each paradigm emerged; cite limitations of predecessors

VI. OPEN PROBLEMS & FUTURE DIRECTIONS GAPS

A. No Systematic Open Problem Listing

Current Status: Conclusion mentions "challenges"; not systematically enumerated

Gap Severity: MEDIUM — Graduate students can't identify research directions

Missing:

Should section enumerate: `` Open Problems in Malware Detection (2024-2025):

Established Challenges (Consensus that these are hard):

- Generalization across datasets & time
- Robustness to obfuscation
- Adversarial evasion
- Scalability to real-world volumes
- Explainability of models
- Cost-benefit of analysis

Emerging Problems (Recently identified as critical):

- Semantic obfuscation resistance
- Test-time adaptation for concept drift
- Adversarial robustness of LLM-based detectors
- Supply-chain malware detection
- Cross-architecture malware detection

Speculative Problems (May or may not be solvable):

- Fundamental limits of adversarial robustness
- Can semantic analysis overcome all obfuscation?
- Will LLMs generalize to unseen code patterns? ``
- **Such enumeration absent**

Recommendation: Add "Open Problems" section; prioritize by impact/feasibility

B. No Discussion of Post-2025 Horizon

Current Status: Conclusion doesn't look beyond 2025

Gap Severity: LOW — Reasonable for 2024 paper; could improve

Missing:

- What's likely to happen in 2026-2027?
- Foundation models becoming dominant?
- Certified robustness becoming standard?
- **Speculation could be valuable**

Recommendation: Add brief "Beyond 2025" section speculating on likely developments

VII. STRUCTURAL & PRESENTATION GAPS

A. No Visual Evolution Timeline

Current Status: No figure showing technique evolution over decade

Gap Severity: MEDIUM — Reduces clarity

Missing Figure: `` 2015 2018 2021 2023 2025 |=====|
=====|=====|=====| Classical ML
Deep Learning Representation Foundation Features CNN/RNN Learning
Models RF/SVM Hybrid GNN/IR Transformers LLMs

Obfuscation: Packing Polymorphism CFG Flattening Semantic Anti-VM Anti-Debug Anti-DBI AI-Powered ``

- No such timeline provided

Recommendation: Add Figure 1: "Technique Evolution 2015-2025"

B. No Maturity Matrix

Current Status: No table showing which techniques are mature vs. emerging

Gap Severity: MEDIUM — Practitioners can't assess readiness

Missing Table: Technique 2015 2018 2021 2023 2025 Status

Classical ML ✓ ✓ ✓ ✓ ✓ Mature Early DL (CNN/RNN) – ✓ ✓ ✓ Legacy
Superseded Hybrid Static+Dynamic – ✓ ✓ ✓ ✓ Mature GNN on CFG – –
Emerg Mature ✓ Mature Binary Lifting + IR – – Emerg Mature ✓
Mature Transformers – – Novel Emerg Mature SOTA LLM-Assisted
– – – Novel Emerg Emerging Few-Shot Meta-Learning – – Emerg
Mature ✓ Mature Concept Drift Handling – – Novel Emerg – Emerging
Adversarial Robustness – – Emerg Mature ✓ Mature

- Not provided in paper

Recommendation: Add Table: "Technique Maturity Matrix"

C. No Prioritized Reading List

Current Status: 106 references; no guidance on what to read first

Gap Severity: MEDIUM — Readers overwhelmed

Missing:

Should provide: `` Prioritized Reading List (2015-2025)

Tier 1: Foundations (Must Read) [33] Gibert et al. (2015) — Taxonomy of features [102] Ye et al. (2017) — Data mining survey [4, 5] Anderson & Roth EMBER (2018) — Benchmark [37, 38] Harang & Rudd SOREL-20M (2021) — Large-scale

Tier 2: Core Methods (Highly Recommended) [30] Galloro et al. (2019) — Evasion techniques [85] Shaukat et al. (2018) — ML/DL techniques [71] Or-Meir et al. (2021) — Dynamic analysis

Tier 3: Modern Approaches (2021-2025) [web:3] BinBert (2024) — Transformers for binary [web:8] Security LLM (2024) — LLM-assisted malware [web:4] CFGExplainer (2022) — GNN explainability

Tier 4: Emerging & Specialized (Select Based on Interest) [web:32] Concept drift (2023-2025) [web:35] Domain adaptation (2024) [web:13] Adversarial robustness (2025) ``

- **Not provided in paper**

Recommendation: Add Appendix: "Prioritized Reading List by Era & Difficulty"

D. No "From-Scratch Survey Writing Guide"

Current Status: Methodology section describes inclusion/exclusion; not how to write survey

Gap Severity: LOW-MEDIUM — Useful for future authors

Missing: Guidance on:

- Literature collection strategy (which databases, search terms)
- Structured extraction (template for analyzing papers)
- Clustering & categorization approach
- Writing workflow (bottom-up vs. top-down)
- LLM vs. manual analysis trade-offs
- Citation verification process
- **Paper doesn't provide this; could be valuable appendix**

Recommendation: Add Appendix: "Guide to Writing Decade-Focused Surveys" (optional but valuable)

E. No Taxonomy Diagram

Current Status: Techniques discussed textually; no visual taxonomy

Gap Severity: MEDIUM — Reduces clarity

Missing:

Should show hierarchy: Malware Detection Approaches (2015–2025) | Legacy (Pre-2015) | Signature-Based | Heuristic Rules | Classical ML Era (2015–2018) | Static Analysis | PE Headers | Opcode n-grams | Strings | Dynamic Analysis | Cuckoo Sandbox | DBI | Classifiers | Random Forest | SVM | Naive Bayes | Deep Learning Era (2018–2021) | CNN on binaries | RNN on API sequences | Hybrid approaches | Structural Analysis Era (2021–2023) | GNN on CFG/DFG | Binary Lifting + IR | Graph Embeddings | Foundation Model Era (2023–2025) | Transformers | LLM-Assisted | Few-Shot Learning

- No such taxonomy diagram provided

Recommendation: Add Figure 2: "Malware Detection Taxonomy (Hierarchical)"

VIII. CRITICAL CONTENT GAPS SUMMARY TABLE

Topic	Coverage	Severity	Gap Type	Fix Effort
Graph Neural Networks	✗ minimal	CRITICAL	Technique	High
Binary Lifting & IR	✗ absent	CRITICAL	Technique	High
Transformers	✗ minimal	CRITICAL	Technique	High
LLM-Assisted Analysis	✗ brief mention	CRITICAL	Technique	High
EMBER2024	✗ absent	CRITICAL	Dataset	Medium
Explainability (SHAP/LIME)	✗ absent	HIGH	Technique	Medium
Concept Drift & Temporal Eval	✗ acknowledged only	HIGH	Evaluation	Medium
Cross-Dataset Generalization	✗ not systematic	HIGH	Evaluation	Medium
Adversarial Robustness	✗ minimal	HIGH	Robustness	High
Practical Scalability	✗ minimal	HIGH	Deployment	Medium
Few-Shot & Meta-Learning	✗ absent	HIGH	Technique	Medium
Tool Reproducibility	✗ not addressed	HIGH	Methodology	Low
Tiered Pipelines	✗ not designed	MEDIUM	Deployment	Medium
Semantic-Level Analysis	✗ weak	MEDIUM	Technique	Medium
Reinforcement Learning	✗ absent	MEDIUM	Technique	Low

Topic	Coverage	Severity	Gap Type	Fix Effort
Visual Timeline	✗ absent	MEDIUM	Presentation	Low
Maturity Matrix	✗ absent	MEDIUM	Presentation	Low
Reading List	✗ absent	MEDIUM	Presentation	Low
Paradigm Stratification	✗ missing table	MEDIUM	Clarity	Low
Failure Mode Analysis	✗ missing	MEDIUM	Understanding	Medium
Open Problems List	✗ not enumerated	MEDIUM	Guidance	Low

FINAL GAP SYNTHESIS

Critical Gaps Blocking "2025 Readiness"

Must Address:

- ✗ Add sections on transformers, LLMs, GNNs (5-8 pages)
- ✗ Update to include EMBER2024 (1-2 pages)
- ✗ Add adversarial robustness section (2-3 pages)
- ✗ Add temporal evaluation framework (2-3 pages)
- ✗ Extend publication cutoff to Dec 2024
- ✗ Add explainability section (1-2 pages)

Should Address:

- ✗ Few-shot & meta-learning (1-2 pages)
- ✗ Practical deployment & scalability (2 pages)
- ✗ Tool reproducibility guidance (1 page)
- ✗ Paradigm stratification table (0.5 page)
- ✗ Visual timeline & maturity matrix (2 figures)
- ✗ Cross-dataset evaluation protocol (1 page)

Would Enhance:

- ✗ Reading list by era (appendix)
- ✗ Open problems enumeration (1 page)
- ✗ Failure modes analysis (optional)
- ✗ From-scratch survey guide (optional appendix)

Estimated Effort:

- **Quick fix** (1 month): Address top 6 critical gaps → 15-20 new pages
- **Moderate revision** (6 weeks): Address top 12 gaps → 25-30 new pages
- **Comprehensive rewrite** (2-3 months): Address all 16 gaps + restructuring

END OF COMPREHENSIVE GAP ANALYSIS

This document maps all identified gaps, missing content, and methodological weaknesses in the paper. Use this alongside the main audit report for complete assessment.